

AI Agent Memory Harness

Persistent memory for long-running coding agents

Team of 4 (Keith · Ken · Brent · Scott) · Two-week sprint · Model-agnostic · Validated on public benchmarks

1 The problem

Long-running coding agents are effectively **stateless across sessions**. Every new session starts cold: the agent has forgotten the fixes it already made, the conventions of the codebase, the dead-ends it already explored, and the environment "gotchas" it already learned. The same mistakes get repeated and the same context gets re-derived at cost.

This hits **smaller, cheaper models hardest**. A frontier model wins partly because it can hold more in its head at once; a smaller model cannot, so it falls behind on long-horizon work. Today the only reliable way to get good long-horizon performance is to pay for the largest model.

There is **no standard, model-agnostic memory layer** that answers the four hard questions well:

- **What** to remember (signal vs. noise), and **when** to write it.
- **Where** to store it (different memory shapes want different stores).
- **How** to retrieve the right thing **fast**, without flooding the context window.
- **How** to keep the store **consistent** — de-duplicated, contradiction-free, pruned.

The opportunity: if an agent can reliably save, retrieve, and reconcile what it learns, a cheaper model should be able to *borrow* the long-horizon advantage of a bigger one.

2 Hypothesis & success criteria

Hypothesis. With the memory harness, **Haiku can close the gap to Opus 4.8** (which runs without memory) on established memory benchmarks.

Success criterion. On **at least two of the four** benchmarks, *Haiku + harness* beats the *Opus 4.8 no-memory* baseline across the four metrics, **without** blowing the efficiency budget (memory adds **< ~10%** context-token overhead).

METRIC	DEFINITION
Recency	Temporal distance of retrieved memory to the current task — is the freshest relevant item ranked first?
Efficiency	Tokens spent per retrieval; target under ~10% overhead.
Relevancy	Semantic similarity of retrieved items to the query — only pull memories that truly relate.
Accuracy	Correctness of agent output, memory-on vs. memory-off.

3 Technical approach

A **pluggable memory harness** with four modules sitting over three storage backends, integrated into the open-source **OpenCode** agent and evaluated against public benchmarks.

Four modules

- **Persistence layer (write)**. Decides what/when/where to save. Tags each item with a timestamp, relevancy score, session tag and provenance; handles versioning and dedup hints.
- **Intelligent router (dispatch)**. Classifies each query and routes it to the *single best* backend rather than fanning out. Rule-based to start, with a learned-classifier upgrade path.
- **Retrieval orchestrator (read)**. Unified query API: calls the router, ranks by `recency × relevancy`, de-duplicates, returns a tight context.
- **Dreaming component (async curate)**. Runs **while other agents sleep**. De-duplicates across backends, resolves contradictions, maintains session governance (must-know / must-do / blacklist), and applies retention + pruning.

Three storage backends (each with its own index)

BACKEND	BEST FOR	INDEX
Markdown + YAML	Human-readable notes, literal recall	Inverted keyword map (keyword → file path)
SQLite + vectors	Semantic similarity search	Dense ANN (HNSW / FAISS)
Graph store	Relationships & conflicts	Typed traversal (node / edge type)

Evaluation

Validated against **existing public benchmarks only** — no home-grown evals. A controlled grid per benchmark runs *Haiku + harness*, *Haiku no-memory*, *Opus 4.8 no-memory*, and *Sonnet no-memory*, holding prompts/tools/task order fixed and logging every trajectory for reproducibility.

4 Scope

In scope

- The memory harness: all four modules, end to end.
- Three storage adapters (Markdown, SQLite-vector, Graph) behind one interface, each indexed.
- The intelligent router (rule-based; learned-classifier hook).
- The async dreaming component: dedup, conflict resolution, session governance, retention/pruning.
- OpenCode integration for systematic memory write/read on each agent step.
- Evaluation harness over the four benchmarks; baselines for Haiku, Opus 4.8 and Sonnet.
- Metric definitions, a reproducible protocol, and a results dashboard.

Out of scope (non-goals)

- **Building our own benchmark or dataset** — we leverage existing public ones.

- **Training or fine-tuning** any model.
- Production hardening: multi-tenant infra, auth, SLAs, a polished UI.
- Distributed / horizontally-scaled deployment — single-node is sufficient for the sprint.
- Validating non-coding agents — the design aims to generalize, but only coding agents are tested now.

5 Team & ownership — who owns what

Four parallel workstreams, locked behind a shared storage interface that is **frozen on Day 3**.

The team: **P1 Keith** (harness, OpenCode) · **P2 Ken** (benchmark) · **P3 Brent** (store, retrieve) · **P4 Scott** (dreaming).

AREA / DELIVERABLE	OWNER	SUPPORTING
Storage interface & memory-item schema	P1 · Keith	P3
Persistence layer (write path, tagging, versioning)	P1 · Keith	—
Retrieval orchestrator (rank · dedup · return)	P1 · Keith	P3
OpenCode integration (write/read on each step)	P1 · Keith	—
Three storage backends + per-backend indexes	P3 · Brent	P1
Intelligent router (rules → learned)	P3 · Brent	P1
Backend performance testing	P3 · Brent	P2
Dreaming worker (dedup, conflict, governance, retention)	P4 · Scott	P1, P3
Memory semantics ("what is good memory")	P4 · Scott	P2
Datasets, loaders & trajectory logging	P2 · Ken	—
Metric definitions & evaluation protocol	P2 · Ken	P4
Baselines + results dashboard	P2 · Ken	—
Final end-to-end integration	All	—

- **P1 · Keith — Harness Architecture & OpenCode Integration.** Owns the critical path: the contracts everyone builds against.
- **P2 · Ken — Benchmarking & Evaluation.** Owns experimental design, metrics, and reproducibility.
- **P3 · Brent — Storage Implementation & Router.** Turns the abstraction into three fast backends + the dispatch layer.
- **P4 · Scott — Dreaming Component & Memory Governance.** Owns the offline engine that keeps memory clean and trustworthy.

6 Two-week timeline

Person 1 · Keith — Harness Architecture & OpenCode Integration

- **Week 1:** Design the three-module abstraction & storage interface. Build the persistence layer (write logic, versioning, metadata tagging). Instrument OpenCode to write memory on each step.
- **Week 2:** Build the retrieval orchestrator (rank → dedup → return). Wire in Brent's router; run an end-to-end task.

Person 2 · Ken — Benchmarking & Evaluation

- **Week 1:** Download & parse MemoryAgentBench, LongMemEval, SWE-ContextBench, SWE-Bench-CL. Build data loaders + trajectory logging. Define the four metrics mathematically.
- **Week 2:** Run baselines (Haiku no-memory, Opus 4.8 no-memory). Build the statistical-testing framework and results dashboard; document the protocol.

Person 3 · Brent — Storage Implementation & Router

- **Week 1:** SQLite + vector pipeline (embedding model, HNSW/FAISS index). Markdown store with YAML frontmatter + inverted index. Graph store schema (Neo4j) with typed traversal index.
- **Week 2:** Adapters for all three backends behind Keith's interface. Implement the router; performance-test each backend.

Person 4 · Scott — Dreaming Component & Memory Governance

- **Week 1:** Deduplication logic (exact / semantic / near-dup). Conflict detection + reconciliation rules (recency, confidence, source). Session-filter & blacklist semantics.
- **Week 2:** Async offline scheduler that runs while agents sleep; must-know / must-do extraction. Retention & pruning; observability logs; integration with Keith & Brent.

Integration — all four, Day 10

Full end-to-end run: Haiku + harness on ≥ 1 benchmark, capturing all four metrics against baselines.

7 Milestones

DAY	MILESTONE
D3	Contract freeze — storage interface + memory-item schema locked; metric definitions agreed.
D5	End of week 1 — three backends write & read in isolation; datasets parsed; persistence writing live.
D8	Integration start — router + orchestrator connected; dreaming worker running against real stores.
D10	Ship — full E2E; four metrics captured vs. baselines.

8 Dependencies & key risk

- **Keith → Brent:** the storage interface must be frozen by end of D3 so adapters can build against it.

- **Brent** → **Keith**: the router lands before the orchestrator can route in week 2.
- **Ken** → **all**: datasets + logging ready by D5 so baselines can start D6.
- **Keith + Brent** → **Scott**: the dreaming worker integrates once real storage exists (D8+).

Top risk is semantic, not structural: defining *what counts as "good memory."* Ken and Scott must align on memory semantics in week 1, or the harness will store everything and retrieve nothing useful.

9 Benchmarks & metrics

BENCHMARK	PRIMARY METRICS	REFERENCE
MemoryAgentBench	Relevancy, Accuracy, conflict resolution	arXiv 2507.05257
LongMemEval	Recency, Accuracy, temporal reasoning	arXiv 2410.10813
SWE-ContextBench	Efficiency, Relevancy, Accuracy	arXiv 2602.08316
SWE-Bench-CL	Relevancy, Accuracy, continual learning	arXiv 2507.00014

Full descriptions, dataset links, and the metric-mapping matrix live on the project site's **Benchmarks** page.

10 Deliverables

- A working, model-agnostic memory harness (4 modules) integrated with OpenCode.
- Three indexed storage backends behind one interface.
- The async dreaming component with governance & pruning.
- A reproducible evaluation harness + baseline numbers on four benchmarks.
- A results scoreboard testing the central hypothesis.
- The project site (GitHub Pages) documenting all of the above.